

广东技术师范大学实验报告

学院：计算机科学学院	专业：软件工程	班级：24 软件工程 X 班	成绩：
姓名：张正浩	学号：	组别：	组员：
实验地点：	实验日期：2024 年 7 月 5 日	指导教师签名：	
预习情况	操作情况	考勤情况	数据处理情况

实验名称：基于 tRPC-Agent 与 OpenCV 的车牌图像识别智能体系统

一、实验目的

- 掌握基于 OpenCV 的车牌图像预处理技术，包括高斯滤波降噪、加权平均灰度化、OTSU 大津阈值二值化、Canny 算子边缘检测以及仿射变换倾斜矫正。
- 掌握车牌图像定位技术，理解数学形态学（腐蚀、膨胀、开运算、闭运算）结合 HSV 颜色空间实现车牌精确定位的方法。
- 掌握基于垂直投影法的车牌字符分割技术，理解波谷-波峰-波谷特征在字符边界检测中的应用。
- 掌握基于 HOG 特征提取 + SVM 支持向量机的车牌字符识别方法，理解 SVM 二分类原理及核函数在高维空间映射中的作用。
- 理解并实践 Agent 工程化架构：基于 tRPC-Agent 框架的 GraphAgent 流水线编排、FunctionTool 工具封装、LLM 二次校验、RAG 知识库检索与跨会话 Memory 记忆召回。
- 理解微服务架构与容器化部署：FastAPI 流式服务化、Spring Boot 管理后台、Vue 3 监控大屏及 Docker Compose 多服务编排。

二、实验内容

本实验基于 OpenCV 计算机视觉库与 tRPC-Agent 智能体框架，构建一套完整的车牌图像自动识别系统 PlateAgent。系统由四大核心模块组成，并扩展了 Agent 智能层和服务化层。

（一）车牌图像预处理模块

对输入的车辆图像进行一系列预处理操作，以提高后续定位和识别的准确率。具体包括：

- 高斯滤波：采用高斯核对图像进行卷积操作，消除高斯噪声，同时较好保留边缘信息。
- 灰度化：使用加权平均法 ($\text{Gray} = 0.299R + 0.587G + 0.114B$) 将彩色图像转为灰度图。
- 二值化：采用 OTSU 大津阈值法自动选取最佳阈值，将灰度图转化为黑白二值图像，分离前景字符与背景。
- Canny 边缘检测：使用 Canny 多级检测算子（高斯平滑 + 梯度计算 + 非极大值抑制 + 双阈值连接）提取图像边缘。
- 仿射变换倾斜矫正：利用 OpenCV 的 `getAffineTransform` 和 `warpAffine` 函数对倾斜车牌进行几何校正。

（二）车牌图像定位模块

采用数学形态学与 HSV 颜色空间相结合的两阶段定位策略：

- 第一阶段（粗定位）：利用数学形态学的腐蚀、膨胀、开运算、闭运算对二值化图像进行处理，通过 `findContours` 提取全部候选轮廓，根据车牌宽高比 ($2:1 \sim 5.5:1$) 进行初步筛选。

- 第二阶段（精定位）：将候选区域的 RGB 颜色空间转换为 HSV 空间，根据 HSV 基本颜色分量范围表计算各矩形中蓝/绿/黄颜色的占有量，最终确定车牌精确区域及车牌颜色。

（三）车牌字符分割模块

采用垂直投影法进行字符分割：对二值化后的车牌图像计算垂直方向投影直方图，利用字符间隙形成的波谷特征定位每个字符的左右边界，将车牌分割为 7 个独立字符图像。同时处理车牌上的铆钉和小圆点干扰。

（四）车牌字符识别模块

采用 HOG（方向梯度直方图）特征提取 + SVM（支持向量机）分类器进行字符识别：

- 汉字分类器：训练 31 个省份简称，各 200 样本。
- 字母数字分类器：训练 24 个英文字母（不含 I、O）+ 10 个数字，各 500 样本。
- LLM 二次校验：当 SVM 置信度低于 0.85 时，自动委托 DeepSeek 大模型进行字符复核，结合混淆字符对照表提升识别准确率。

（五）Agent 智能层

基于 tRPC-Agent 框架构建 GraphAgent 流水线，将上述四大模块封装为 10+ FunctionTool，通过条件路由实现 SVM → LLM 自适应校验；集成 ChromaDB 向量数据库构建车辆黑名单 RAG 知识库，支持语义检索和多轮对话查询；通过 Redis Session 实现跨会话 Memory 记忆召回。

（六）服务化层

通过 FastAPI + SSE 实现流式识别服务，支持实时推送预处理→定位→分割→识别的各阶段进度事件；Spring Boot + JWT 构建管理后台 CRUD 接口；Vue 3 +

ECharts 构建监控大屏实时展示识别统计；Docker Compose 编排 7 个微服务容器（Nginx + FastAPI + Spring Boot + Redis + MySQL + ChromaDB + RabbitMQ）。

三、实验步骤

步骤一：环境搭建与配置

1. 安装 Python 3.11，配置虚拟环境，安装依赖包（tRPC-Agent、OpenCV、NumPy、scikit-learn、FastAPI、ChromaDB、Redis 等）。
2. 配置 DeepSeek API Key，设置 .env 环境变量文件。
3. 安装并启动 Redis 7.x、MySQL 8.0、ChromaDB 等中间件服务。
4. 创建 Spring Boot 3.3 项目，配置 Spring Security + JWT + JPA 依赖。
5. 创建 Vue 3 项目，安装 ECharts、Axios、Pinia、Vue Router 等依赖。

步骤二：OpenCV 工具函数封装

1. 编写 `tool_gaussian_blur()`：使用 `cv2.GaussianBlur()` 实现 5×5 高斯核卷积降噪。
2. 编写 `tool_grayscale()`：使用 `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` 实现加权平均灰度化。
3. 编写 `tool_binarize_otsu()`：使用 `cv2.threshold()` 配合 `cv2.THRESH_OTSU` 实现自动阈值二值化。
4. 编写 `tool_edge_detect_canny()`：使用 `cv2.Canny()` 实现 Canny 边缘检测（双阈值 50/150）。

5. 编写 `tool_affine_correct()`: 使用 `cv2.getAffineTransform()` + `cv2.warpAffine()` 实现仿射变换矫正。
6. 编写 `tool_morphology_locate()`: 使用 `cv2.getStructuringElement()` + `cv2.morphologyEx()` 实现形态学处理, `cv2.findContours()` 提取候选轮廓。
7. 编写 `tool_color_locate()`: `cv2.cvtColor()` 转 HSV, `cv2.inRange()` 筛选颜色区域, 宽高比筛选。
8. 编写 `tool_vertical_projection()`: `np.sum()` 计算垂直投影直方图, 通过波谷检测分割字符边界。
9. 编写 `tool_svm_predict()`: 加载训练好的 SVM 模型, HOG 特征提取后 `predict()` 输出字符 + 置信度。
10. 编写 `tool_llm_verify()`: 调用 DeepSeek API 对低置信度字符进行二次校验。
11. 编写 `tool_search_blacklist()`: ChromaDB 向量检索黑名单。

步骤三: tRPC-Agent GraphAgent 编排

1. 定义 `PlateState` 状态对象 (`TypedDict`), 包含 `image_path`、`preprocess_output`、`locate_output`、`segment_chars`、`recognize_chars`、`final_plate` 等字段。
2. 实现 `preprocess_node`: 串联高斯滤波→灰度化→二值化→Canny→仿射矫正 5 个预处理步骤。
3. 实现 `locate_node`: 串联形态学粗定位→HSV 精定位, 输出车牌区域图像。
4. 实现 `segment_node`: 垂直投影分割, 输出 7 个字符图像列表。

5. 实现 `recognize_node`: `asyncio.gather` 并行 SVM 预测 7 个字符，标记低置信度字符。
6. 实现 `llm_verify_node`: 对 `needs_verify=True` 的字符调用 LLM 复核，支持 `safe_llm_call` 三层容错（重试 + 超时 + 降级）。
7. 实现 `format_output_node`: 拼接最终车牌号，查询黑名单，生成可读响应。
8. 配置条件路由: `needs_llm_verify=True` → `llm_verify_node`; 否则跳过直接进入 `format_output`。

步骤四：FastAPI 服务化

1. 创建 FastAPI 应用，配置 CORS 中间件和 OpenTelemetry 自动追踪。
2. POST `/api/chat`: LlmAgent 多轮对话接口，支持 SSE 流式输出。
3. POST `/api/recognize`: 单张图像识别接口，调用 GraphAgent 流水线。
4. GET `/api/health`: 健康检查接口。

步骤五：Spring Boot 管理后台

1. 创建 `PlateRecord` 实体类，映射 `plate_records` 表（车牌号、置信度、识别方式、黑名单状态等字段）。
2. 编写 `PlateRecordRepository`，提供分页查询、状态统计、黑名单筛选等 JPA 方法。
3. 编写 `PlateRecordService`，实现业务逻辑和今日统计聚合。

4. 编写 PlateController, 暴露 RESTful API: GET /api/records (分页查询)、GET /api/records/stats/today (今日统计)、GET /api/records/stats/hourly (小时分布)。

5. 配置 Spring Security + JWT: JwtUtil 生成/验证 Token, JwtAuthFilter 拦截请求, AuthController 处理登录。

步骤六: Vue 3 监控大屏

1. 实现 Dashboard.vue: 6 个统计卡片 (总识别次数、成功/部分成功/失败、黑名单命中、平均置信度) + ECharts 柱状图 (今日每小时分布) + ECharts 饼图 (状态占比) + 最近识别记录表格。

2. 实现 Records.vue: 分页表格展示所有识别记录, 支持车牌号搜索和状态筛选, 一键切换黑名单视图。

3. 实现 Login.vue: JWT 登录页, Token 存入 localStorage。

4. 配置 Vue Router 路由守卫, 未登录自动跳转登录页。

步骤七: Docker Compose 多服务编排

1. 编写 docker-compose.yml: 定义 Nginx、FastAPI、Spring Boot、Redis、MySQL、ChromaDB、RabbitMQ 共 7 个服务。

2. 编写 nginx.conf: 配置反向代理规则, /api/chat 和 /api/recognize 代理到 FastAPI, /api/auth 和 /api/records 代理到 Spring Boot, / 返回 Vue 静态资源。

3. 编写 Dockerfile.fastapi: 基于 python:3.11-slim, 安装 OpenCV 系统依赖, pip install 项目依赖。

4. 编写 Dockerfile.springboot: Maven 多阶段构建 (maven:3.9 -> eclipse-temurin:17-jre) 。
5. 编写 Dockerfile.nginx: Node 构建 Vue 静态资源 + nginx:alpine 提供 Web 服务。
6. 编写 init.sql: 自动创建 plate_agent 数据库和 plate_records 表。
7. 执行 docker-compose up -d 启动全部服务。

四、实验问题及原因

1. SVM 字符识别中的混淆问题

在字符识别过程中，部分形近字符（如“粤”与“鄂”、“B”与“8”、“D”与“0”、“2”与“Z”等）容易出现误识别，导致置信度低于 0.85。原因在于 HOG 特征对笔画细节的区分能力有限，且训练样本覆盖的场景不够丰富。解决方案：引入 LLM 二次校验机制，结合混淆字符对照表进行上下文推理，有效提升了识别准确率。

2. 复杂光照条件下的定位失败

在强光反光、夜间或阴影不均匀的场景下，HSV 颜色定位算法效果下降，车牌颜色分量范围漂移导致定位失败或误定位。原因在于 HSV 颜色分量阈值是固定经验值，对光照变化鲁棒性不足。解决方案：可引入直方图均衡化预处理改善光照不均，或使用深度学习目标检测模型（如 YOLO）替代传统颜色定位。

3. 黏连字符分割困难

当车牌出现污渍、锈蚀或拍摄角度导致字符黏连时，垂直投影法的波谷特征不再明显，导致分割失败。原因在于投影法依赖字符间隙的灰度特征，对物理损伤敏感。解决方案：可采用连通域分析或基于聚类的 K-Means 分割作为备选方案。

4. DeepSeek API 调用超时与降级

在高峰期或网络不稳定时，LLM 二次校验请求可能出现超时（>10s）或 429 限流错误。原因在于免费 API 的并发限制和网络延迟。解决方案：使用 tenacity 库实现指数退避重试（最多 3 次，总超时 30s），超时后自动降级为 SVM 原始结果。

5. Docker 容器间网络通信

Spring Boot 容器调用 FastAPI 容器时，因 localhost 指向自身容器而非宿主机导致连接被拒。原因在于 Docker Compose 中各容器有独立网络栈。解决方案：使用服务名（如 fastapi:8000）代替 localhost 进行容器间通信，通过 Nginx 统一对外暴露 API。